

Formally Verifying the KZG Polynomial Commitment Scheme

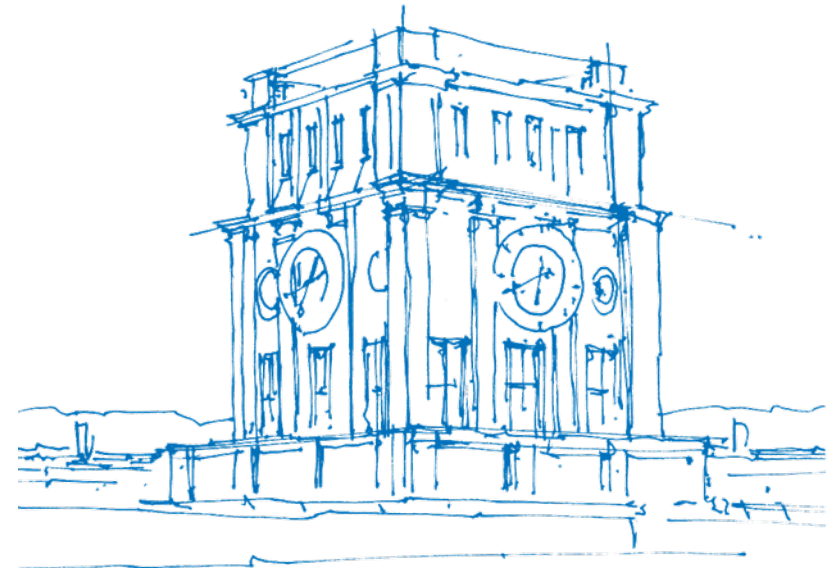
Tobias Rothmann

Technical University of Munich

School of Computation, Information and Technology

Chair for Logic and Verification

28. Mai 2024



TUM Uhrenturm

Outline

- Motivation
- Polynomial Commitment Schemes
- KZG
- Formal Verification

Motivation

What is formal verification?

What is formal verification?

- providing a formal proof

What is formal verification?

- providing a formal proof
⇒ providing a machine-checkable proof

What is formal verification?

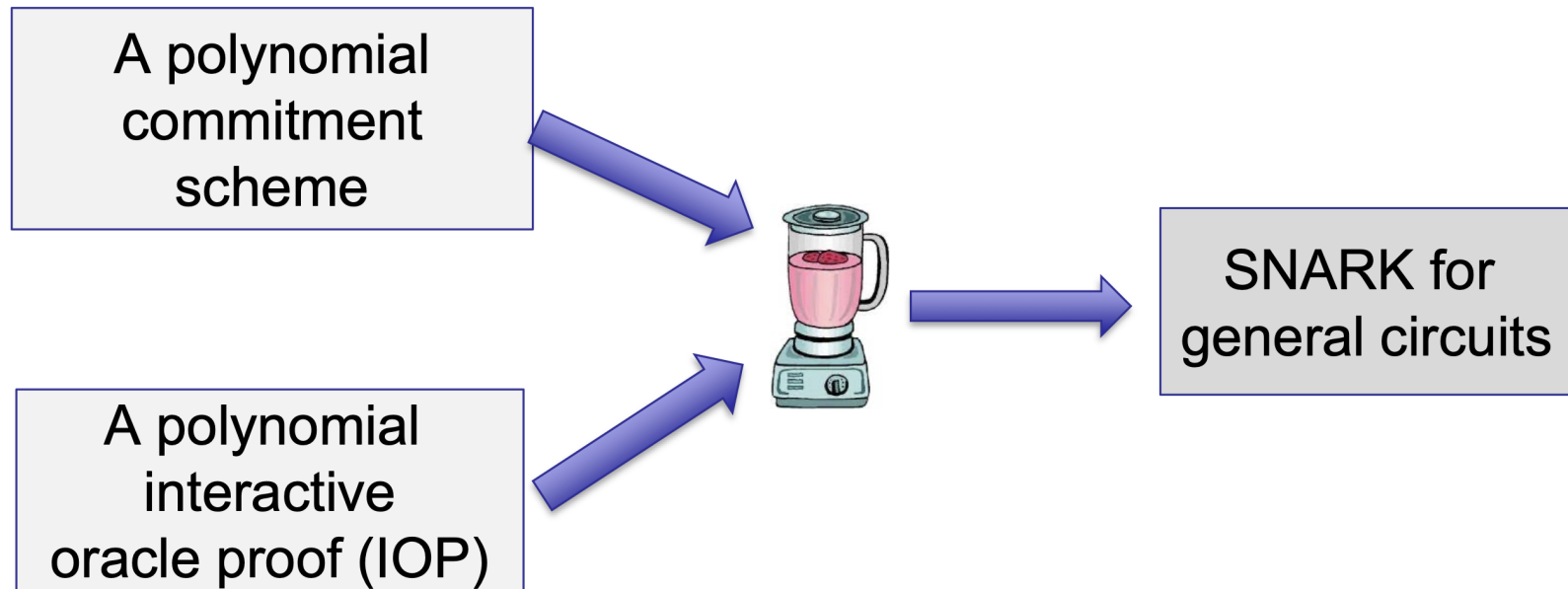
- providing a formal proof
 - ⇒ providing a machine-checkable proof
- undecidable

What is formal verification?

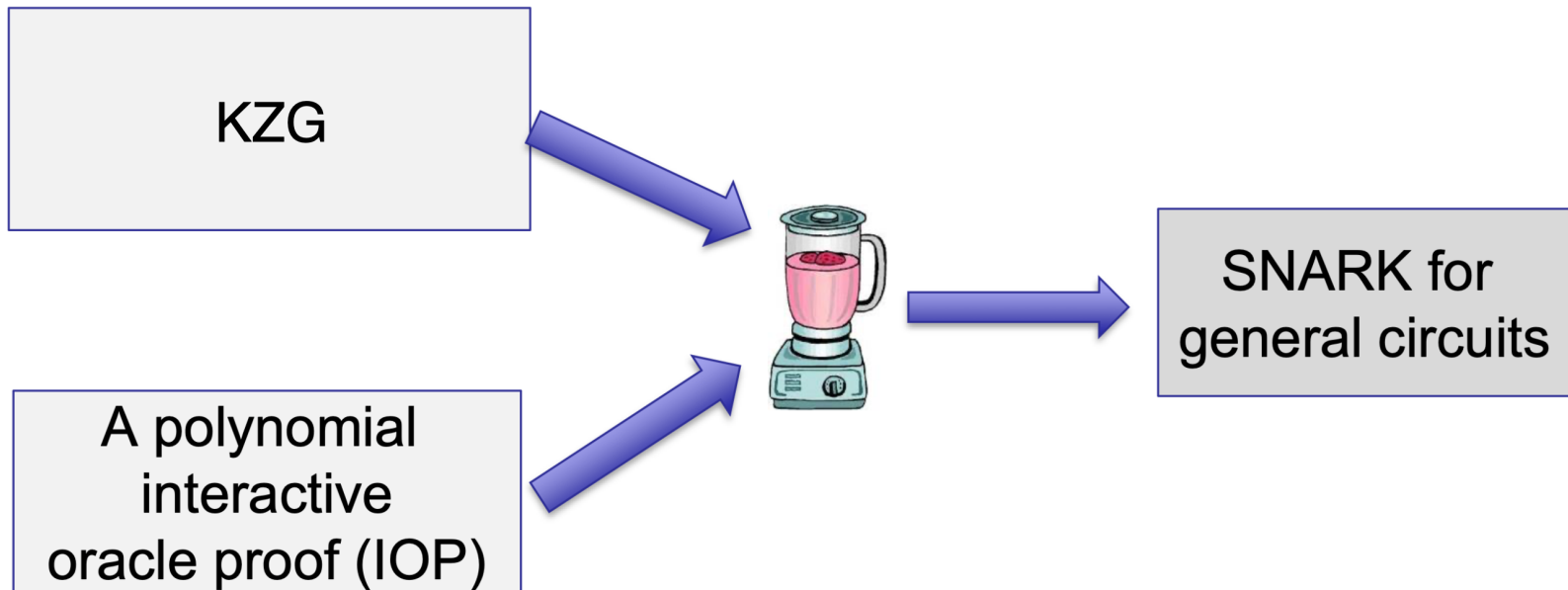
- providing a formal proof
⇒ providing a machine-checkable proof
- undecidable
⇒ *Pain in the A***

Why formally verify the KZG?

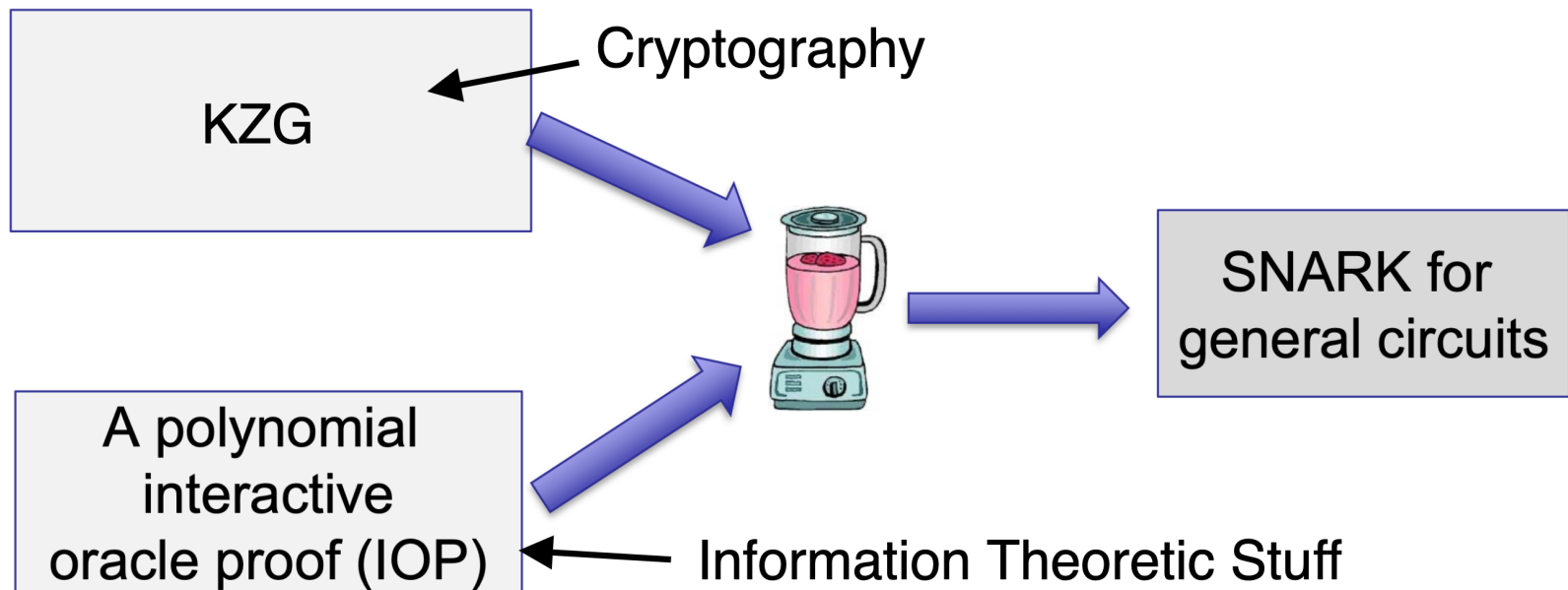
Why formally verify the KZG?



Why formally verify the KZG?

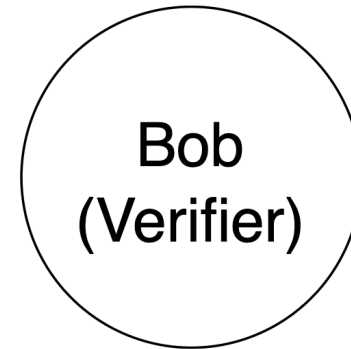
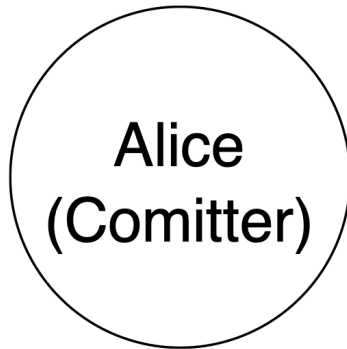


Why formally verify the KZG?

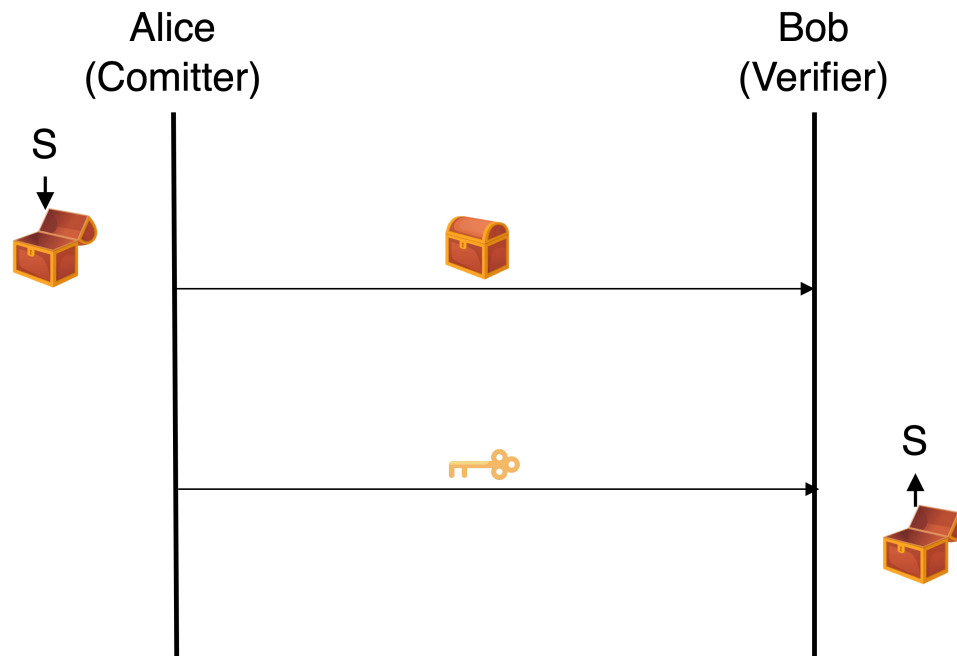


Polynomial Commitment Scheme

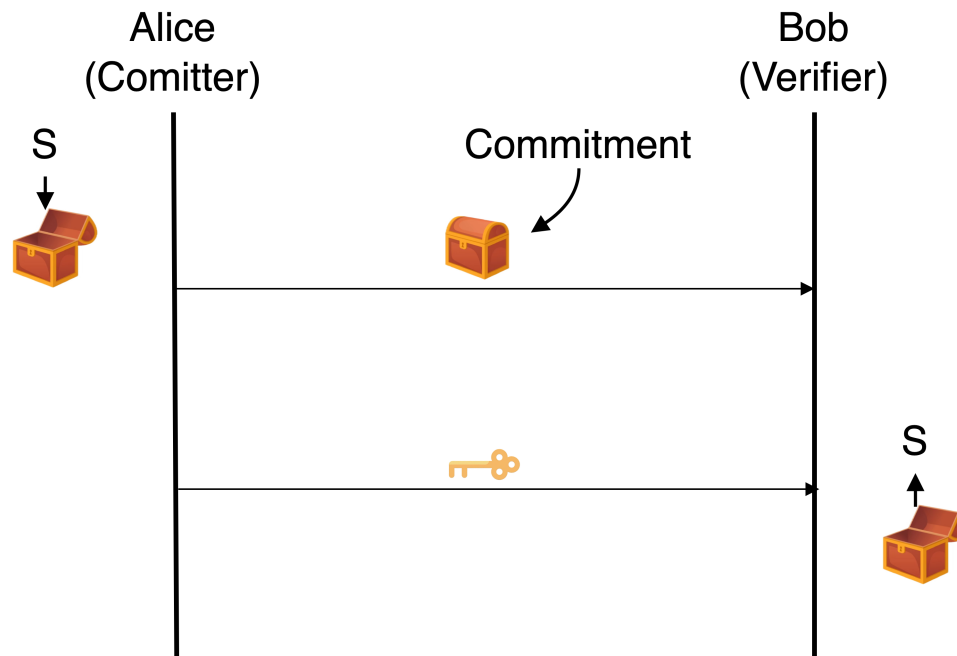
Commitment Scheme



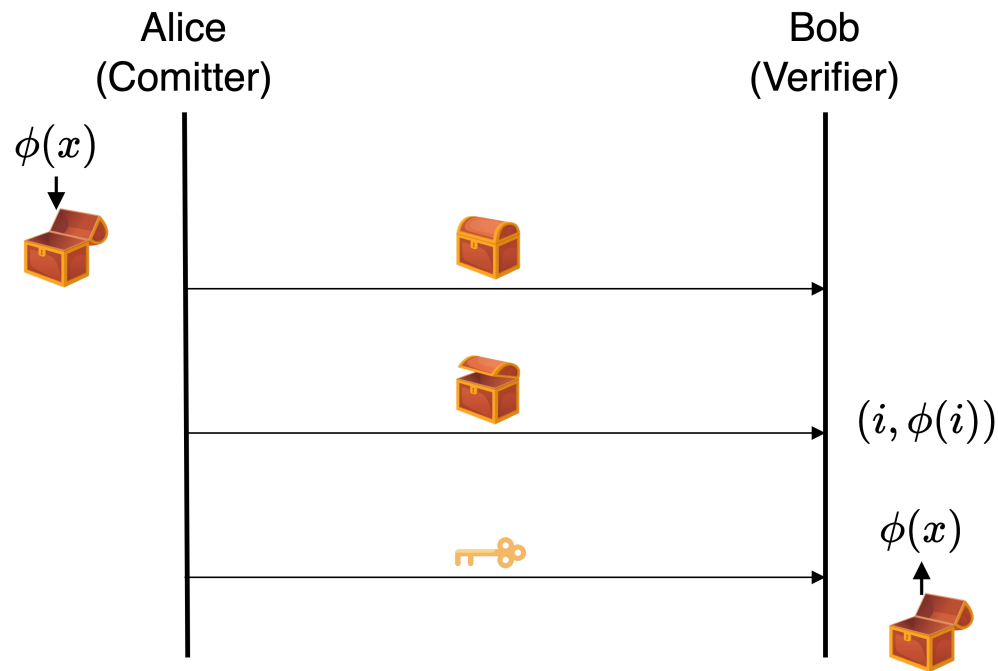
Commitment Scheme



Commitment Scheme

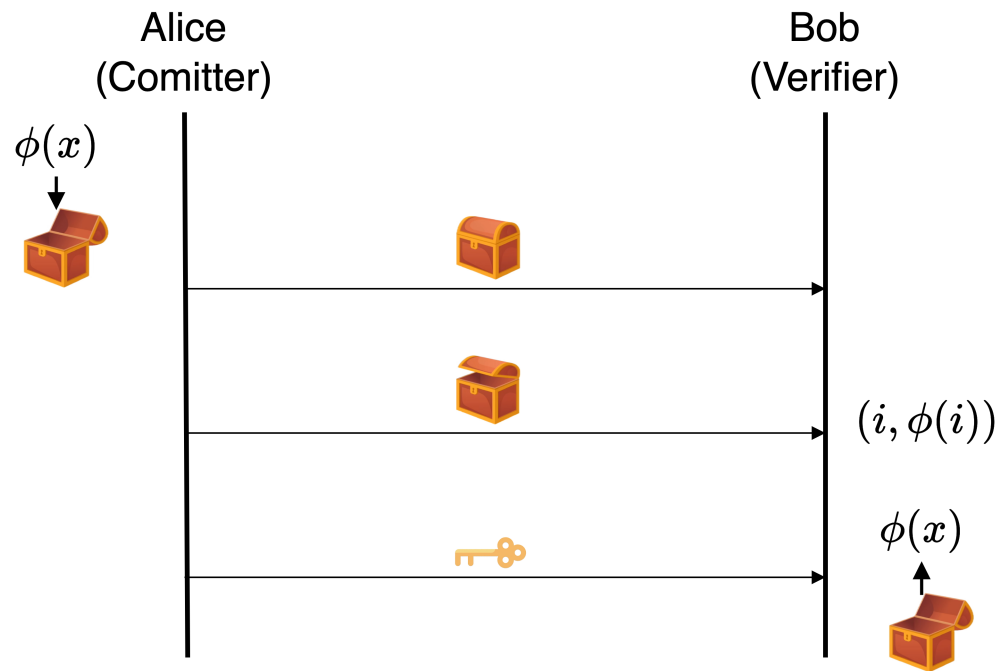


Commitment Scheme

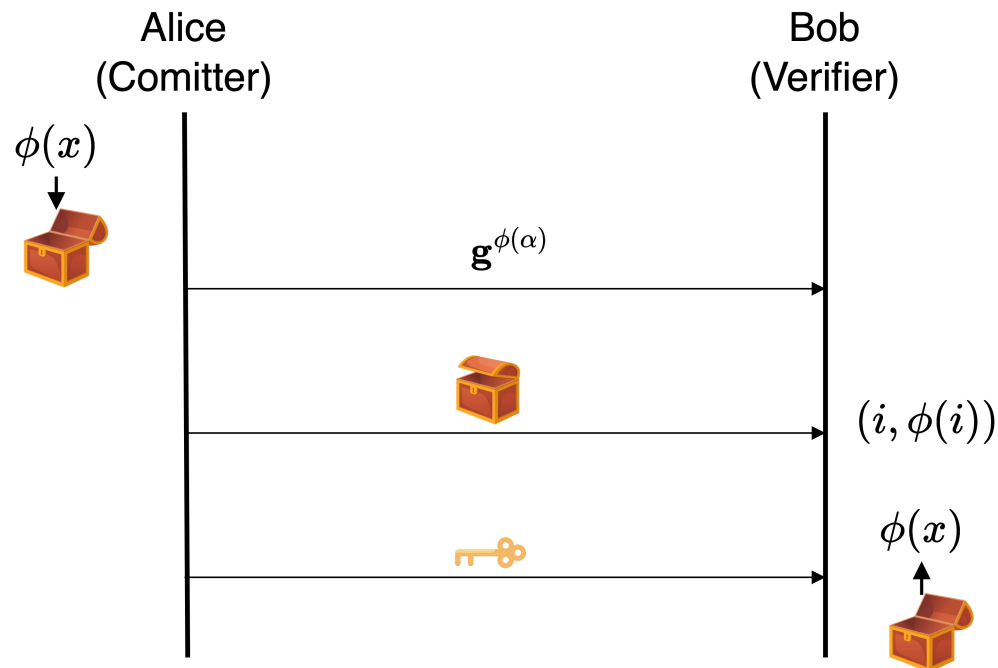


The KZG

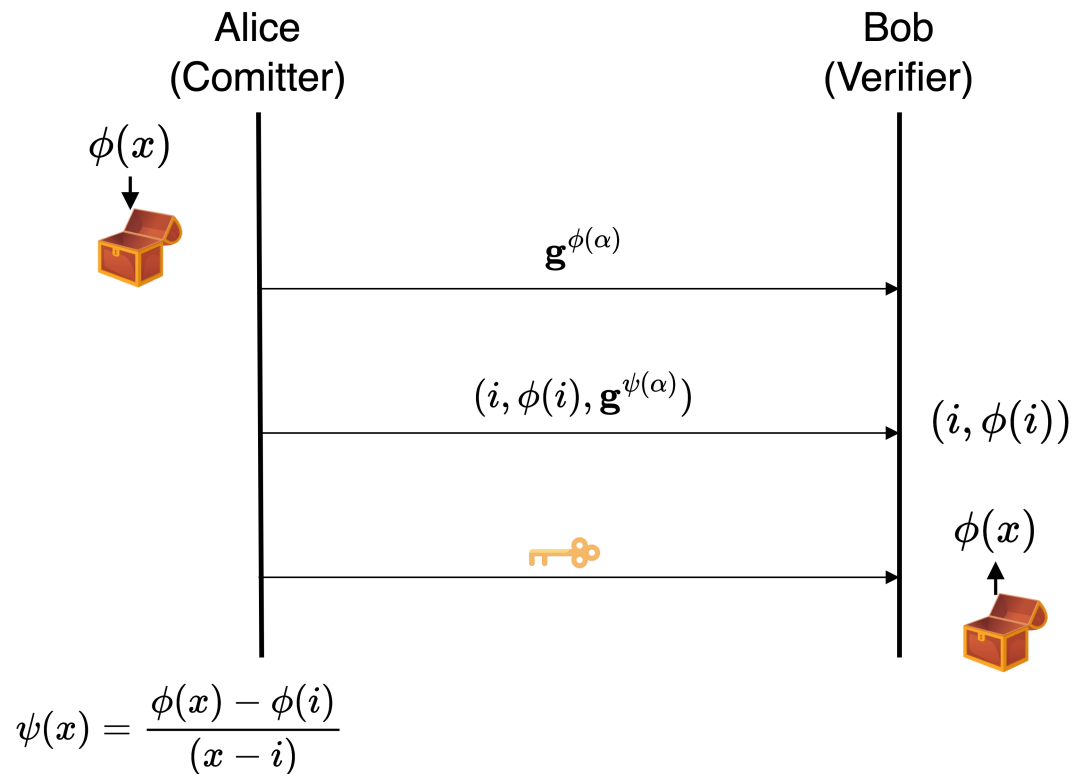
KZG - Polynomial Commitment Scheme



KZG - Polynomial Commitment Scheme



KZG - Polynomial Commitment Scheme



KZG - how to prove a peek? (Intuition)

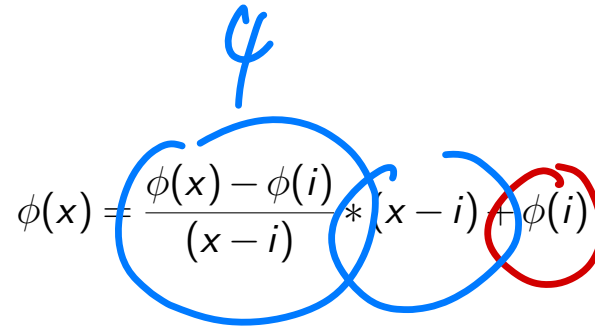
Commitment - $\mathbf{g}^{\phi(\alpha)}$

Peek - $(i, \phi(i), \mathbf{g}^{\psi(\alpha)})$

KZG - how to prove a peek? (Intuition)

Commitment - $\mathbf{g}^{\phi(\alpha)}$

Peek - $(i, \phi(i), \mathbf{g}^{\psi(\alpha)})$

$$\phi(x) = \frac{\phi(x) - \phi(i)}{(x - i)} * (x - i) + \phi(i)$$


KZG - how to prove a peek? (Intuition)

Commitment - $\mathbf{g}^{\phi(\alpha)}$

Peek - $(i, \phi(i), \mathbf{g}^{\psi(\alpha)})$

$$\phi(x) = \psi(x) * (x - i) + \phi(i)$$

KZG - how to prove a peek? (Intuition)

Commitment - $\mathbf{g}^{\phi(\alpha)}$

Peek - $(i, \phi(i), \mathbf{g}^{\psi(\alpha)})$

$$x=i$$

$$\phi(x) = \underbrace{\psi(x) * (x - i)}_{\phi(i)} + \phi(i)$$

$$\phi(i) = \phi(i)$$

KZG - how to prove a peek? (Intuition)

Commitment - $\mathbf{g}^{\phi(\alpha)}$

Peek - $(i, \phi(i), \mathbf{g}^{\psi(i)})$

$$g^a$$

$$G \times G \rightarrow G_T$$

$$e(g^a, g^b) = g^{a \cdot b}$$

$$\phi(x) = \psi(x) * (x - i) + \phi(i)$$

$\Downarrow$ Schwarz-Zippel

$$\phi(\alpha) = \psi(\alpha) * (\alpha - i) + \phi(i)$$

$$g^{\psi(\alpha)} \alpha$$

KZG - how to prove a peek? (Intuition)

Commitment - $\mathbf{g}^{\phi(\alpha)}$

Peek - $(i, \phi(i), \mathbf{g}^{\psi(\alpha)})$

$$\phi(x) = \psi(x) * (x - i) + \phi(i)$$

$\Downarrow$ Schwarz-Zippel

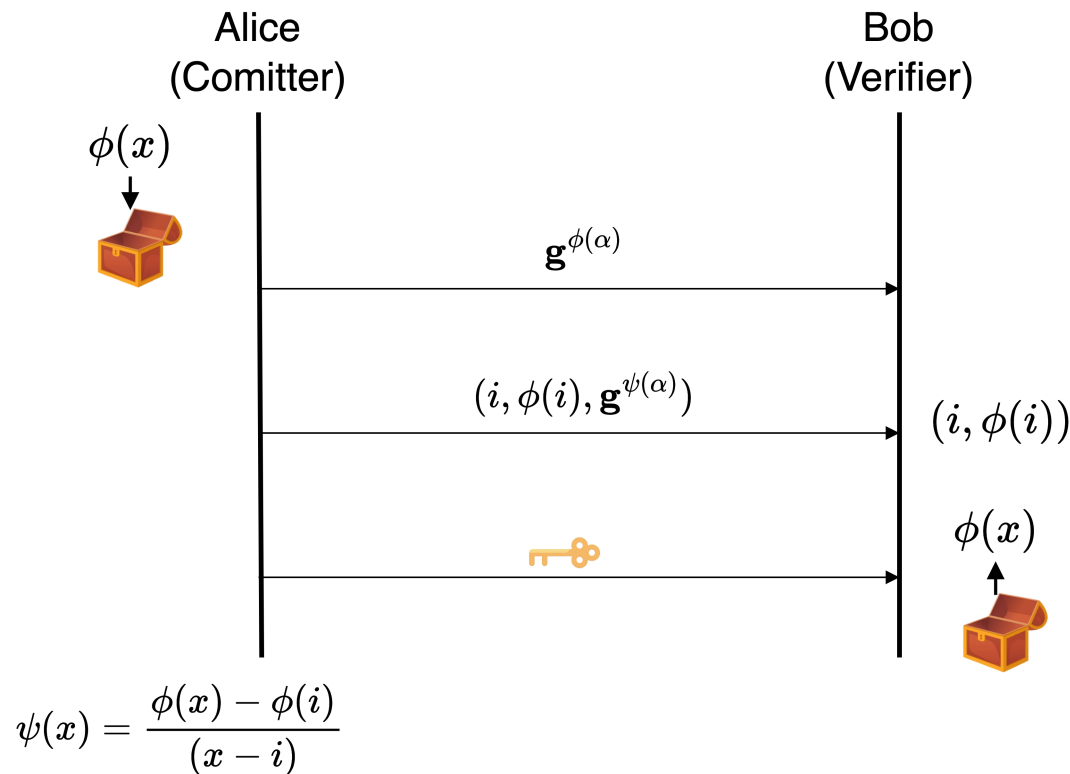
$$\phi(\alpha) = \psi(\alpha) * (\alpha - i) + \phi(i)$$

$\Downarrow$ Pairing

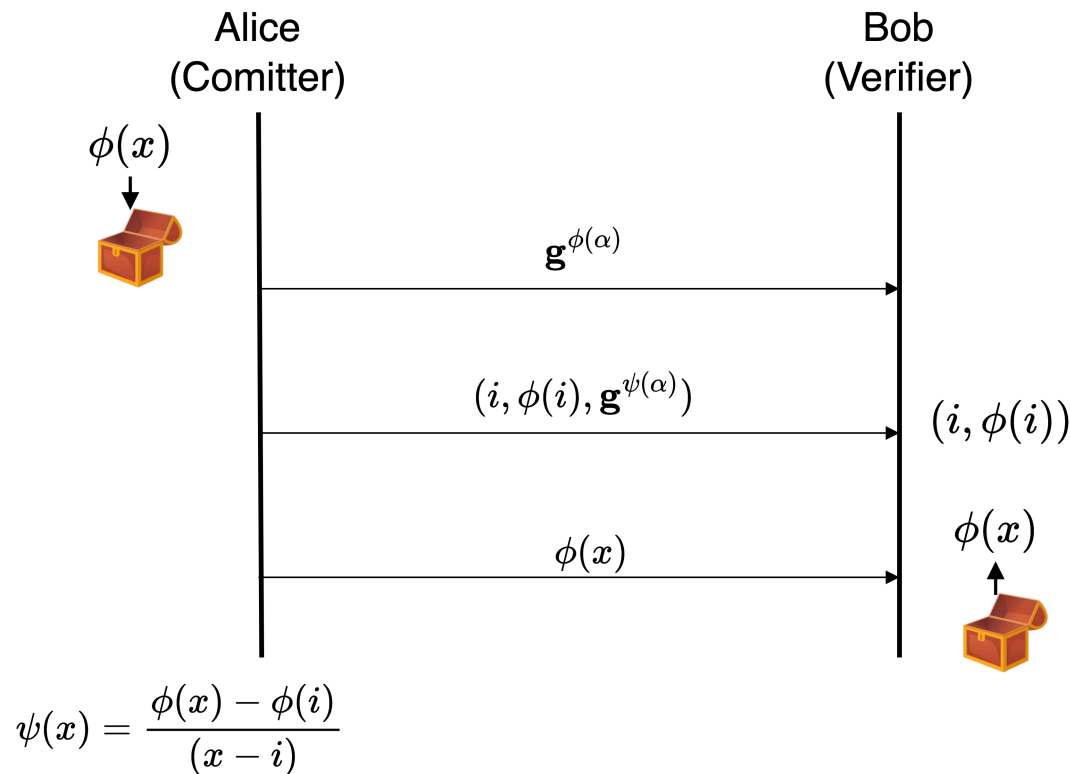
$$e(\mathbf{g}^{\phi(\alpha)}, \mathbf{g}) = e\left(\mathbf{g}^{\psi(\alpha)}, \frac{\mathbf{g}^{\alpha}}{\mathbf{g}^i}\right) \cdot e(\mathbf{g}^{\phi(i)}, \mathbf{g})$$

Disclaimer: This was soley intuition.
Not security proofs.

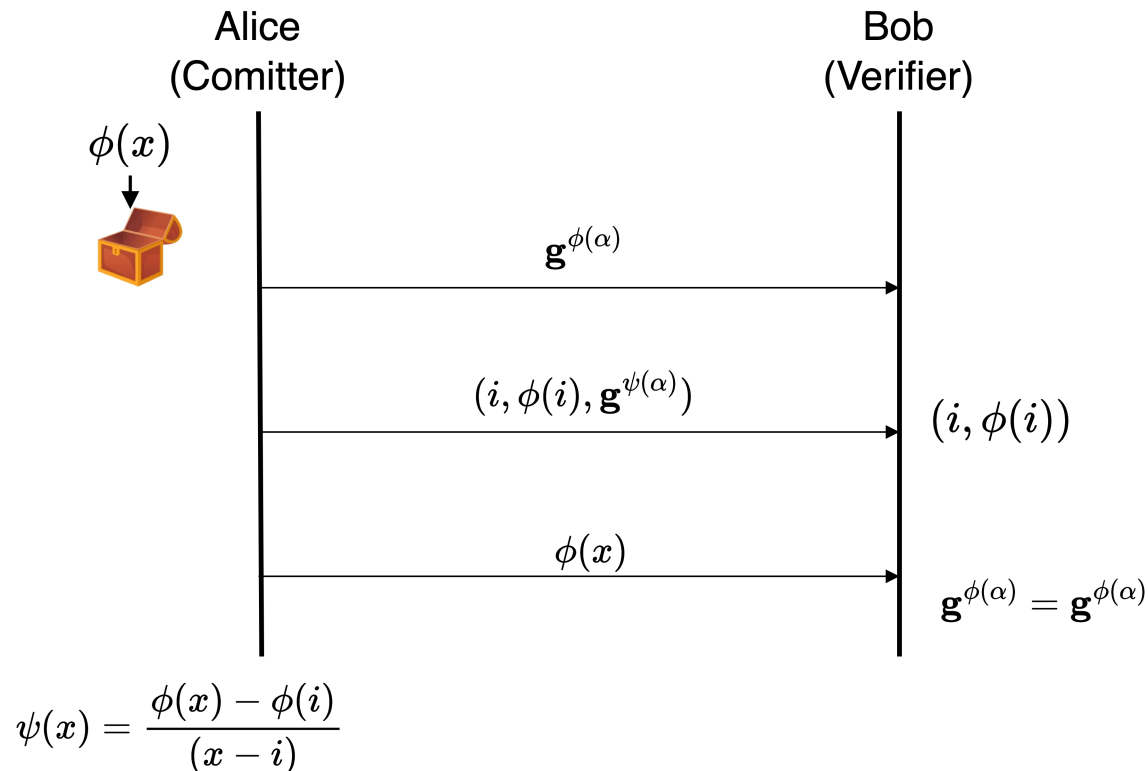
KZG - Polynomial Commitment Scheme



KZG - Polynomial Commitment Scheme



KZG - Polynomial Commitment Scheme



Formal Verification

What do we formally verify? (Highlevel)

What do we formally verify? (Highlevel)

Security proofs for the KZG:

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynomial binding

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynomial binding $\rightarrow$ Alice cannot change $\phi(x)$ in 

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynomial binding $\rightarrow$ Alice cannot change $\phi(x)$ in 
- evaluation binding

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynomial binding $\rightarrow$ Alice cannot change $\phi(x)$ in 
- evaluation binding $\rightarrow$ Alice cannot reveal  $:= (i, \phi(i))$ and  $' := (i, \phi(i)')$ with  $\neq$  $' \equiv \phi(i) \neq \phi(i)'$

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynomial binding $\rightarrow$ Alice cannot change $\phi(x)$ in 
- evaluation binding $\rightarrow$ Alice cannot reveal  $:= (i, \phi(i))$ and  $' := (i, \phi(i)')$ with  $\neq$  $' \equiv \phi(i) \neq \phi(i)'$
- hiding

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynomial binding $\rightarrow$ Alice cannot change $\phi(x)$ in 
- evaluation binding $\rightarrow$ Alice cannot reveal  $:= (i, \phi(i))$ and  $' := (i, \phi(i)')$ with  $\neq$  $' \equiv \phi(i) \neq \phi(i)'$
- hiding $\rightarrow$ Bob cannot infer $\phi(x)$ from 

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynomial binding $\rightarrow$ Alice cannot change $\phi(x)$ in 
- evaluation binding $\rightarrow$ Alice cannot reveal  $:= (i, \phi(i))$ and  $' := (i, \phi(i)')$ with  $\neq$  $' \equiv \phi(i) \neq \phi(i)'$
- hiding $\rightarrow$ Bob cannot infer $\phi(x)$ from 
- knowledge soundness

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynomial binding $\rightarrow$ Alice cannot change $\phi(x)$ in 
- evaluation binding $\rightarrow$ Alice cannot reveal  $:= (i, \phi(i))$ and  $' := (i, \phi(i)')$ with  $\neq$  $' \equiv \phi(i) \neq \phi(i)'$
- hiding $\rightarrow$ Bob cannot infer $\phi(x)$ from 
- knowledge soundness $\rightarrow$ Alice cannot reveal points without knowing $\phi(x)$ in 

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynom

- evaluatic



- hiding $\rightarrow$ Bob cannot infer $\phi(x)$ from $\phi(x)$

- knowledge soundness $\rightarrow$ Alice cannot reveal points without knowing $\phi(x)$ in

Standard PCS

$(i, \phi(i)')$ with

Security Properties

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynomial

- evaluation



- hiding —, ~~one cannot infer $\phi(x)$ from~~

- knowledge sound

Standard PCS

$(i, \phi(i)')$ with

Security Properties

SNARK precondition

knowing $\phi(x)$ in 

What do we formally verify? (Highlevel)

Security proofs for the KZG:

- polynomial binding $\rightarrow$ Alice cannot change $\phi(x)$ in 
- evaluation binding $\rightarrow$ Alice cannot reveal  $:= (i, \phi(i))$ and  $' := (i, \phi(i)')$ with  $\neq$  $' \equiv \phi(i) \neq \phi(i)'$
- hiding $\rightarrow$ Bob cannot infer $\phi(x)$ from 
- knowledge soundness $\rightarrow$ Alice cannot reveal points without knowing $\phi(x)$ in 

What do we formally verify?

Games against adversary Eve

What do we formally verify?

Games against adversary Eve

→ games as probabilistic algorithms

What do we formally verify?

Games against adversary Eve

- games as probabilistic algorithms
- Eve modeled as arbitrary probabilistic algorithm

What do we formally verify?

Games against adversary Eve

- games as probabilistic algorithms
- Eve modeled as arbitrary probabilistic algorithm
- Goal: prove

$$Pr(\text{Eve wins game}) \approx 0$$

What do we formally verify?

Games against adversary Eve

- games as probabilistic algorithms
- Eve modeled as arbitrary probabilistic algorithm
- Goal: prove

$$Pr(\text{Eve wins game}) \approx 0$$

Example Hiding Game:

$$\left(\begin{array}{l} C \leftarrow \text{📦}, \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(C, wtnss), \\ : \phi = \phi' \end{array} \right)$$

How do we prove this?

How do we prove this?

⇒ Reduction from Hard Problem

How do we prove this?

⇒ Reduction from Hard Problem

Example Discrete Log:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z}_p \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

How do we prove this?

⇒ Reduction from Hard Problem

Example Discrete Log:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z}_p \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

remember: Hiding game:

$$\left(\begin{array}{l} C \leftarrow \text{📦}, \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(C, wtnss), \\ : \phi = \phi' \end{array} \right)$$

How do we prove this?

⇒ Reduction from Hard Problem

Example Discrete Log:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z}_p \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

remember: Hiding game:

$$\left(\begin{array}{l} C \leftarrow \text{📦} := \mathbf{g}^{\phi(\alpha)}, \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(C, wtnss), \\ : \phi = \phi' \end{array} \right)$$

How do we prove this?

Reduction:

⇒ Reduction from Hard Problem

Example Discrete Log:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z}_p \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

remember: Hiding game:

$$\left(\begin{array}{l} C \leftarrow \text{📦}, \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(C, wtnss), \\ : \phi = \phi' \end{array} \right)$$

How do we prove this?

⇒ Reduction from Hard Problem

Reduction:

let $I = [(y, \mathbf{g}^b), \dots]$

Example Discrete Log:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z}_p \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

remember: Hiding game:

$$\left(\begin{array}{l} C \leftarrow \text{📦}, \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(C, wtnss), \\ : \phi = \phi' \end{array} \right)$$

How do we prove this?

⇒ Reduction from Hard Problem

Example Discrete Log:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z}_p \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

remember: Hiding game:

$$\left(\begin{array}{l} C \leftarrow \text{📦}, \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(C, wtnss), \\ : \phi = \phi' \end{array} \right)$$

Reduction:

let $I = [(y, \mathbf{g}^b), \dots]$

$$\left(\begin{array}{l} \mathbf{g}^{\phi(\alpha)} \leftarrow \text{interpolate}((x, \mathbf{g}^a) \# I), \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(\mathbf{g}^{\phi(\alpha)}, wtnss), \\ : \phi = \phi', \\ \phi(x) = \alpha \end{array} \right)$$

How do we prove this?

⇒ Reduction from Hard Problem

Example Discrete Log:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z}_p \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

remember: Hiding game:

$$\left(\begin{array}{l} C \leftarrow \text{📦}, \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(C, wtnss), \\ : \phi = \phi' \end{array} \right)$$

Reduction:

let $I = [(y, \mathbf{g}^b), \dots]$

$$\left(\begin{array}{l} \mathbf{g}^{\phi(\alpha)} \leftarrow \text{interpolate}((x, \mathbf{g}^a) \# I), \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(\mathbf{g}^{\phi(\alpha)}, wtnss), \\ : \phi = \phi', \\ \phi(x) \end{array} \right)$$

⇒ Eve efficient ⇒ DL broken

How do we prove this?

⇒ Reduction from Hard Problem

Example Discrete Log:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z}_p \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

remember: Hiding game:

$$\left(\begin{array}{l} C \leftarrow \text{📦}, \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(C, wtnss), \\ : \phi = \phi' \end{array} \right)$$

Reduction:

let $I = [(y, \mathbf{g}^b), \dots]$

$$\left(\begin{array}{l} \mathbf{g}^{\phi(\alpha)} \leftarrow \text{interpolate}((x, \mathbf{g}^a) \# I), \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(\mathbf{g}^{\phi(\alpha)}, wtnss), \\ : \phi = \phi', \\ \phi(x) \end{array} \right)$$

⇒ Eve efficient ⇒ DL broken

⇒ Is this reduction cryptographically sound?

How do we prove this?

⇒ Reduction from Hard Problem

Example Discrete Log:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z}_p \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

remember: Hiding game:

$$\left(\begin{array}{l} C \leftarrow \text{📦}, \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(C, wtnss), \\ : \phi = \phi' \end{array} \right)$$

Reduction:

let $I = [(y, \mathbf{g}^b), \dots]$

$$\left(\begin{array}{l} \mathbf{g}^{\phi(\alpha)} \leftarrow \text{interpolate}((x, \mathbf{g}^a) \# I), \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(\mathbf{g}^{\phi(\alpha)}, wtnss), \\ : \phi = \phi', \\ \phi(x) \end{array} \right)$$

⇒ Eve efficient ⇒ DL broken

⇒ Is this reduction cryptographically sound?

NO!

How do we prove this?

⇒ Reduction from Hard Problem

Example Discrete Log:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z}_p \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

remember: Hiding game:

$$\left(\begin{array}{l} C \leftarrow \text{📦}, \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(C, wtnss), \\ : \phi = \phi' \end{array} \right)$$

Reduction:

let $I = [(y, \mathbf{g}^b), \dots]$

$$\left(\begin{array}{l} \mathbf{g}^{\phi(\alpha)} \leftarrow \text{interpolate}((x, \mathbf{g}^a) \# I), \\ wtnss \leftarrow \text{📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \text{ 📦} \dots, \\ \phi' \leftarrow \text{Eve}(\mathbf{g}^{\phi(\alpha)}, wtnss), \\ : \phi = \phi', \\ \phi(x) \end{array} \right)$$

⇒ Eve efficient ⇒ DL broken

⇒ Is this reduction cryptographically sound?

NO!

⇒ "no adversary can break hiding for arbitrary polynomials"

vs "for arbitrary polynomials no adversary can break hiding"

A Better Way - Sequences of Games [Shoup04]

Transformation via Game-hops

A Better Way - Sequences of Games [Shoup04]

Transformation via Game-hops $\Rightarrow$ show security game = hardness game for some adversary

$$\text{Security Game} = \left(\begin{array}{c} \dots \end{array} \right) = \left(\begin{array}{c} \dots \end{array} \right) \leq \left(\begin{array}{c} \dots \end{array} \right) = \dots = \left(\begin{array}{c} \dots \end{array} \right) = \text{Hardness Assumption Game}$$

A Better Way - Sequences of Games [Shoup04]

Transformation via Game-hops $\Rightarrow$ show security game = hardness game for some adversary

$$\text{Security Game} = \left(\begin{array}{c} \dots \end{array} \right) = \left(\begin{array}{c} \dots \end{array} \right) \leq \left(\begin{array}{c} \dots \end{array} \right) = \dots = \left(\begin{array}{c} \dots \end{array} \right) = \text{Hardness Assumption Game}$$

1. Bridging step:

A Better Way - Sequences of Games [Shoup04]

Transformation via Game-hops $\Rightarrow$ show security game = hardness game for some adversary

$$\text{Security Game} = \left(\dots \right) = \left(\dots \right) \leq \left(\dots \right) = \dots = \left(\dots \right) = \text{Hardness Assumption Game}$$

1. Bridging step:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right) = \left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a' = a \end{array} \right)$$

A Better Way - Sequences of Games [Shoup04]

Transformation via Game-hops $\Rightarrow$ show security game = hardness game for some adversary

$$\text{Security Game} = \left(\dots \right) = \left(\dots \right) \leq \left(\dots \right) = \dots = \left(\dots \right) = \text{Hardness Assumption Game}$$

1. Bridging step:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right) = \left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a' = a \end{array} \right)$$

2. Failure-event:

A Better Way - Sequences of Games [Shoup04]

Transformation via Game-hops $\Rightarrow$ show security game = hardness game for some adversary

$$\text{Security Game} = \left(\begin{array}{c} \dots \end{array} \right) = \left(\begin{array}{c} \dots \end{array} \right) \leq \left(\begin{array}{c} \dots \end{array} \right) = \dots = \left(\begin{array}{c} \dots \end{array} \right) = \text{Hardness Assumption Game}$$

1. Bridging step:

$$\left(\begin{array}{c} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right) = \left(\begin{array}{c} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a' = a \end{array} \right)$$

2. Failure-event:

$$\left(\begin{array}{c} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right) \approx \left(\begin{array}{c} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^{\frac{1}{a}}) \\ : \frac{1}{a} = a' \end{array} \right)$$

A Better Way - Sequences of Games [Shoup04]

Transformation via Game-hops $\Rightarrow$ show security game = hardness game for some adversary

$$\text{Security Game} = \left(\dots \right) = \left(\dots \right) \leq \left(\dots \right) = \dots = \left(\dots \right) = \text{Hardness Assumption Game}$$

1. Bridging step:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right) = \left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a' = a \end{array} \right)$$

3. Over-estimation:

2. Failure-event:

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right) \approx \left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^{\frac{1}{a}}) \\ : \frac{1}{a} = a' \end{array} \right)$$

A Better Way - Sequences of Games [Shoup04]

Transformation via Game-hops $\Rightarrow$ show security game = hardness game for some adversary

$$\text{Security Game} = \left(\begin{array}{c} \dots \end{array} \right) = \left(\begin{array}{c} \dots \end{array} \right) \leq \left(\begin{array}{c} \dots \end{array} \right) = \dots = \left(\begin{array}{c} \dots \end{array} \right) = \text{Hardness Assumption Game}$$

1. Bridging step:

$$\left(\begin{array}{c} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right) = \left(\begin{array}{c} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a' = a \end{array} \right)$$

3. Over-estimation:

$$\left(\begin{array}{c} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right) \leq \left(\begin{array}{c} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \end{array} \right)$$

2. Failure-event:

$$\left(\begin{array}{c} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right) \approx \left(\begin{array}{c} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^{\frac{1}{a}}) \\ : \frac{1}{a} = a' \end{array} \right)$$

How do we formally VERIFY this?

⇒ turn formal proof into machine checkable proof:

How do we formally VERIFY this?

⇒ turn formal proof into machine checkable proof:

```
TRY do {  
  a ← sample_uniform p;  
  a' ← Eve(ga);  
  return_spmf (a = a')  
} ELSE return_spmf False
```

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

How do we formally VERIFY this?

⇒ turn formal proof into machine checkable proof:

```

TRY do {
  a ← sample_uniform p;
  a' ← Eve(ga);
  return_spmf (a = a')
} ELSE return_spmf False

```

$$\left(\begin{array}{l} a \xleftarrow{\$} \mathbb{Z} \\ a' \leftarrow \text{Eve}(\mathbf{g}^a) \\ : a = a' \end{array} \right)$$

⇒ prove game-hop correctness in Higher Order Logic (HOL) (machine checked)

Summary

Summary

- first formalization of a Polynomial Commitment Scheme (PCS)

Summary

- first formalization of a Polynomial Commitment Scheme (PCS)
- show all typical cryptographic security properties

Summary

- first formalization of a Polynomial Commitment Scheme (PCS)
- show all typical cryptographic security properties
- first step to formally verify KZG-based SNARKs

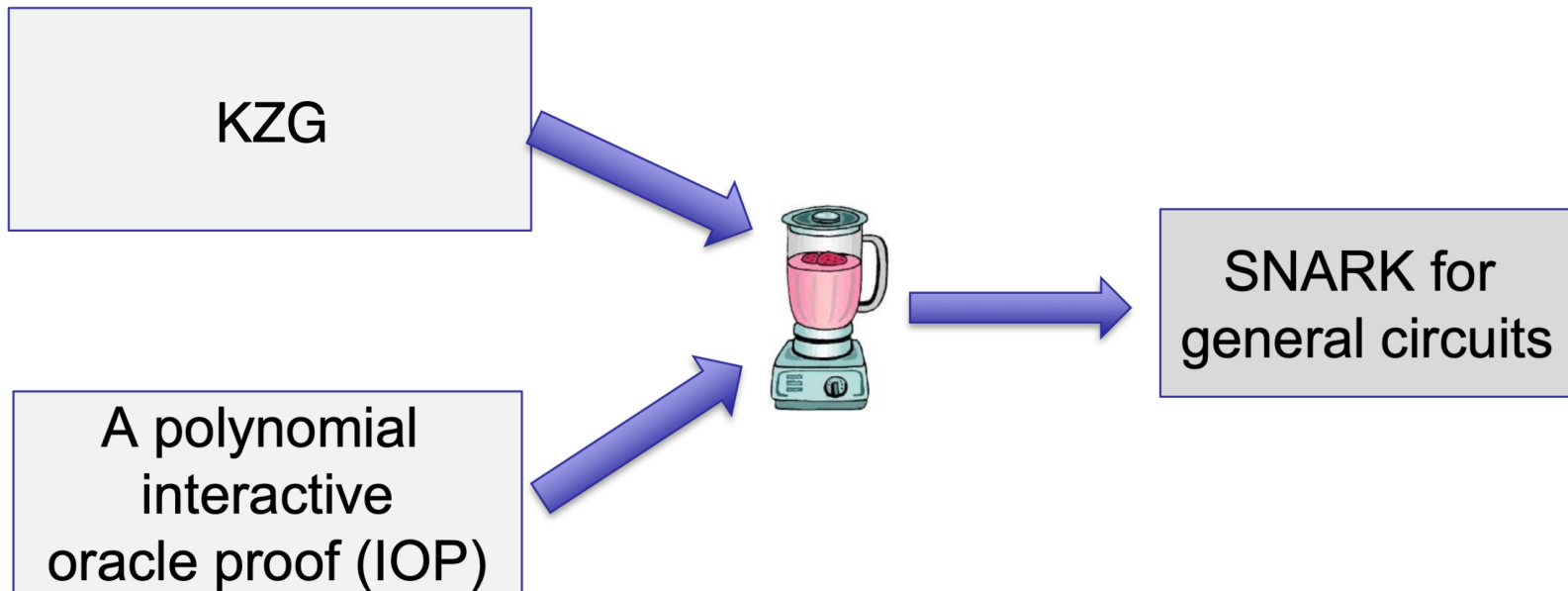
Open Problems

Open Problems

- generate formally verified Go implementation

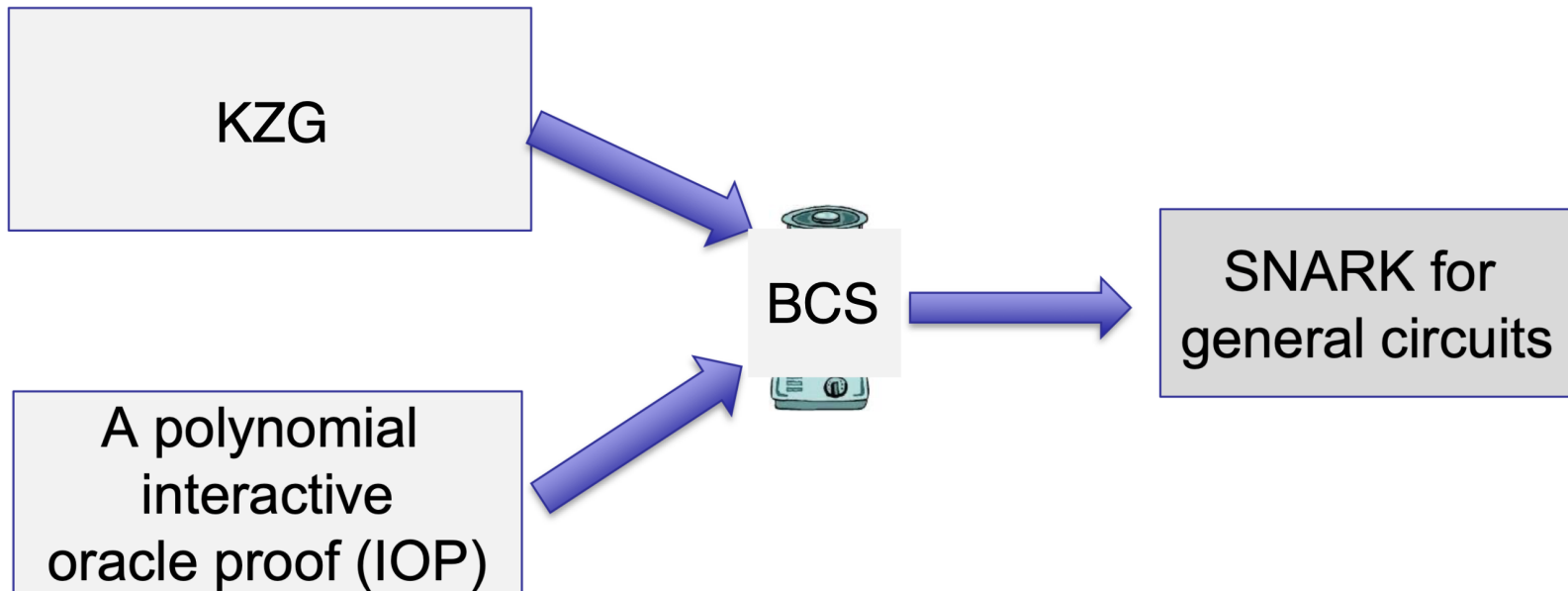
Open Problems

- generate formally verified Go implementation
- SNARKs:



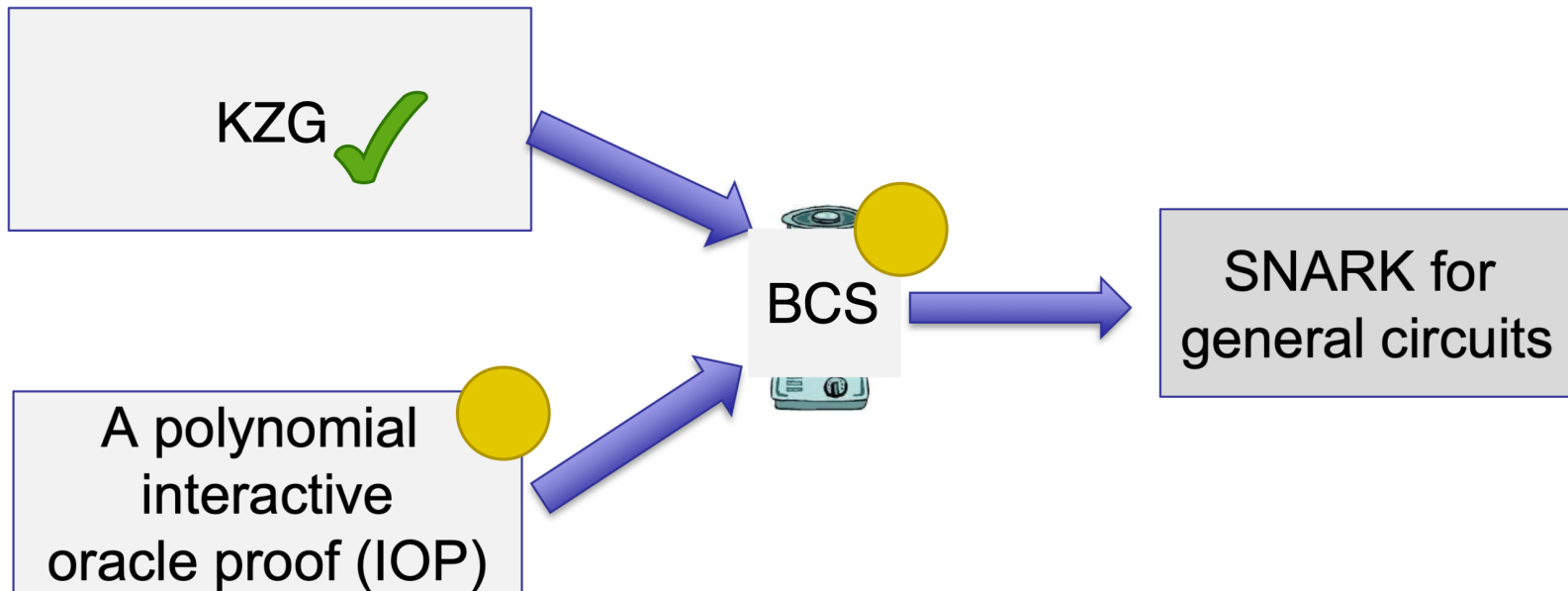
Open Problems

- generate formally verified Go implementation
- SNARKs:



Open Problems

- generate formally verified Go implementation
- SNARKs:



References

- Isabelle formalization of the KZG:
<https://github.com/tobias-rothmann/KZG-Polynomial-Commitment-Scheme>
- Kate, Zaverucha, Goldberg - Constant-Size Commitments to Polynomials and Their Applications
<https://www.iacr.org/archive/asiacrypt2010/6477178/6477178.pdf>
- Shoup - Sequences of games: a tool for taming complexity in security proofs
<https://eprint.iacr.org/2004/332>
- KZG-based SNARKs - Plonk, Halo, Sonic, Marlin, Nova:
<https://eprint.iacr.org/2019/953.pdf>, <https://eprint.iacr.org/2020/1536>,
<https://eprint.iacr.org/2019/099>, <https://eprint.iacr.org/2019/1047>, <https://eprint.iacr.org/2021/370>
- Ben-Sasson, Chiesa, Spooner - Interactive Oracle Proofs
<https://www.iacr.org/archive/tcc2016b/99850156/99850156.pdf>

Formally Verifying the KZG Polynomial Commitment Scheme

Tobias Rothmann

Technical University of Munich

School of Computation, Information and Technology

Chair for Logic and Verification

28. Mai 2024



TUM Uhrenturm

Tobias Rothmann
Technical University of Munich
School of Computation, Information and Technology
Chair for Logic and Verification
28. Mai 2024

